WEEK 10

Measurement, Collapse, and Reading Output

20, 22 October · Objectives 2, 4 *Tuesday 20 October: Fall Break — no class*

Measurement is where a quantum computation becomes a classical result, and it is the only lossy step in the pipeline. This week makes the projection formalism concrete and connects it to the counts dictionary you actually get back.

Projective measurement

Measuring observable $\hat{A} = \Sigma_m a_m |m\rangle\langle m|$ on $|\psi\rangle$:

```
Outcome:      a_m      with probability   P(m) = |⟨m|ψ⟩|²
Post-state:   |m⟩      (the projector applied and renormalized)
```

More generally, with projector $\hat{P}_m$ onto the eigenspace of a_m:

$$P(m) = \langle\psi|\hat{P}_m|\psi\rangle \qquad |\psi'\rangle = \hat{P}_m|\psi\rangle \,/\, \sqrt{P(m)}$$

Two properties define the operation:

Repeatability. Measure again immediately and you get the same result with certainty — the state is now an eigenstate. This is what makes measurement a *projection* rather than a random scramble.

Irreversibility. $\hat{P}_m$ is not unitary. $\hat{P}_m^2 = \hat{P}_m$ (idempotent), and it has zero eigenvalues, so it has no inverse. The information in the discarded components is gone.

> **Analogy boundary — collapse as a state-machine commit.** The parallel is strong: observation transitions the system to a definite state, and prior alternatives become unreachable — exactly your audit-trail write. Two boundaries. First, the *timing* of collapse is interpretation-dependent (Week 3); the formalism specifies outcome statistics, not a moment of transition. Second, your commit selects among branches that were separately real; quantum measurement selects among branches that were interfering, and could have cancelled. There was no fact about which branch was "actual" beforehand — Bell's theorem forecloses that reading.

Basis choice is the experiment

Hardware measures in the computational (Z) basis only. Measuring anything else means **rotating the state so your target basis becomes the Z basis:**

```
    Measure X:   apply H,        then measure Z
   Measure Y:   apply S† then H, then measure Z
```

```
# (X) on |+⟩ : rotate X-basis into Z-basis, then measure.
qc = QuantumCircuit(1, 1)
qc.h(0)                 # prepare |+⟩
qc.h(0)                 # basis rotation: X-eigenbasis -> Z-eigenbasis
qc.measure(0, 0)
# Result: '0' with certainty. |+⟩ is a definite state in the X basis.
```

The same |+⟩ gives 50/50 in Z and certainty in X. Superposition is basis-relative — Week 4's claim, now as an executable demonstration.

Reading the counts dictionary

```
{'00': 512, '11': 488}
```

Three facts to keep straight:

Little-endian. `'01'` means qubit 1 = 0, qubit 0 = 1. Reversed from the natural reading, and the single most common bug in student code.

Counts are samples, not probabilities. With N shots, the standard error on each estimated probability is √(p(1−p)/N). At N = 1024 and p = 0.5, that is ±1.6% — so a 48/52 split is entirely consistent with a fair distribution.

Classical registers are indexed independently of qubits. `measure(2, 0)` puts qubit 2's result in classical bit 0. Multiple registers print space-separated, most-significant first.

```
from qiskit.result import marginal_counts
marginal_counts(counts, indices=[0])     # distribution over qubit 0 alone
```

Partial measurement

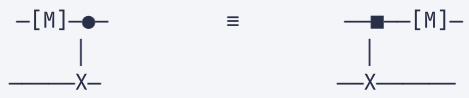
Measuring one qubit of an entangled pair collapses both:

```
 qc = QuantumCircuit(2, 1)
qc.h(0); qc.cx(0, 1)      # |Φ⁺⟩
qc.measure(0, 0)          # measure ONLY qubit 0
# Qubit 1 is now determined: |0⟩ or |1⟩, matching qubit 0, though never measured.
```

Formally, the post-measurement state of the unmeasured qubit follows from applying $P_m \otimes I$ and renormalizing. This is the mechanism behind teleportation and behind the syndrome extraction of Week 16 — measuring *ancillas* to learn about data qubits without touching them.

Deferred measurement

A useful theorem: any measurement followed by a classically-controlled operation can be replaced by a coherent controlled operation, with all measurements moved to the end.



This matters practically. Mid-circuit measurement with feed-forward requires fast classical control that many devices lack or implement slowly; the deferred form runs on any device. Theoretically, it means measurement is never *necessary* mid-computation — a fact used constantly in proofs.

Interpreting noisy output

Expected for a Bell state; observed on hardware:

```
 ideal:     {'00': 512, '11': 512}
hardware:  {'00': 470, '11': 468, '01': 45, '10': 41}
```

The diagnostic sequence, in order:

1. **Baseline readout error.** Run an empty circuit preparing $|00\rangle$. Any non-`00` counts are pure readout error, correctable by inverting the confusion matrix.
2. **Compare against a noise model.** `AerSimulator.from_backend(backend)` reproduces the device's error rates in simulation. If hardware is much worse, suspect calibration drift.
3. **Check depth after transpilation.** SWAP insertion can multiply two-qubit gate count several-fold, and each CNOT carries ~1% error.

```
 from qiskit_aer import AerSimulator
noisy = AerSimulator.from_backend(backend)
print(noisy.run(transpile(qc, noisy), shots=4096).result().get_counts())
```

Key takeaways

- Measurement applies a projector: outcome with probability $\langle\psi|P_m|\psi\rangle$, state renormalized to the eigenspace. Repeatable, and not unitary.
- Hardware measures in Z only; other bases require an explicit rotation first (H for X, S†H for Y).
- Counts are samples with standard error $\sqrt{p(1-p)/N}$; Qiskit bitstrings are little-endian.
- Measuring one qubit of an entangled pair determines the other without touching it.
- Deferred measurement lets any mid-circuit measurement move to the end — essential on devices without fast feed-forward.

Self-check

Q1 *(Objective 2)* — Measuring $|+\rangle$ in the Z basis gives 50/50; in the X basis it gives a certain outcome. A colleague concludes the qubit "had a hidden value revealed by the right question." Explain why this reading fails, referring to a specific experimental result.

Q2 *(Objective 4)* — Your two-qubit circuit returns `{'01': 1000}`. You expected qubit 0 to be 0 and qubit 1 to be 1. Did the circuit work? Explain, and give the code that resolves the ambiguity.

Check your answers

A1 — The hidden-value reading proposes that the qubit carries pre-existing answers for every possible measurement, and that measuring selects which to reveal. This is precisely the **local hidden variable** hypothesis, and it is experimentally refuted.

For a single qubit the reading is not immediately contradictory — you can construct a hidden-variable model reproducing all single-qubit statistics. The refutation requires **entangled pairs and Bell's inequality**. If each particle carried definite values for all measurement directions, the CHSH correlation would satisfy $S \leq 2$. Quantum mechanics predicts $2\sqrt{2}$, and the loophole-free experiments

of 2015 at Delft, NIST, and Vienna confirmed the violation with both the locality and detection loopholes closed simultaneously. Aspect, Clauser, and Zeilinger received the 2022 Nobel Prize for this line of work.

So the accurate statement is not that measurement reveals a hidden value, but that it **produces** a value that did not exist beforehand — with statistics fixed by the state, and a correlation structure no assignment of pre-existing values can reproduce. Note the qubit *is* in a definite state, $|+\rangle$; what it lacks is a definite Z-value. Those are different claims, and keeping them apart is most of the conceptual work.

A2 — You cannot tell from the bitstring alone, and the ambiguity is **little-endian ordering**.

Qiskit prints qubit 0 **rightmost**. So `'01'` means qubit 1 = 0, qubit 0 = 1 — the opposite of what you expected. If your intent was qubit 0 = 0 and qubit 1 = 1, the correct bitstring is `'10'`, and this result indicates a genuine bug.

Resolve it by never parsing bitstrings positionally:

```
 # Marginalize onto specific qubits — order-explicit, no manual indexing.
from qiskit.result import marginal_counts
print(marginal_counts(counts, indices=[0]))   # qubit 0 alone
print(marginal_counts(counts, indices=[1]))   # qubit 1 alone

# Or make the mapping explicit with named registers.
from qiskit import QuantumRegister, ClassicalRegister, QuantumCircuit
qr = QuantumRegister(2, 'q'); cr = ClassicalRegister(2, 'c')
qc = QuantumCircuit(qr, cr)
qc.measure(qr[0], cr[0])
qc.measure(qr[1], cr[1])
```

The practice worth adopting: assert on marginals in tests rather than on full bitstrings. Positional string parsing is where endianness bugs hide, and they survive review because the string *looks* right.